

Cryptography and Network Security

Fifth Edition
by William Stallings

Chapter 5

“Cryptography and Network Security”, 5/e, by William Stallings, Chapter Chapter 5
– “Advanced Encryption Standard”.

Chapter 5 -Advanced Encryption Standard

"It seems very simple."

"It is very simple. But if you don't know what the key is it's virtually indecipherable."

—Talking to Strange Men, Ruth Rendell

Intro quote.

Origins

- ❑ clear a replacement for DES was needed
 - have theoretical attacks that can break it
 - have demonstrated exhaustive key search attacks
- ❑ can use Triple-DES - but slow, has small blocks
- ❑ US NIST issued call for ciphers in 1997
- ❑ 15 candidates accepted in Jun 98
- ❑ 5 were shortlisted in Aug-99
- ❑ Rijndael was selected as the AES in Oct-2000
- ❑ issued as FIPS PUB 197 standard in Nov-2001

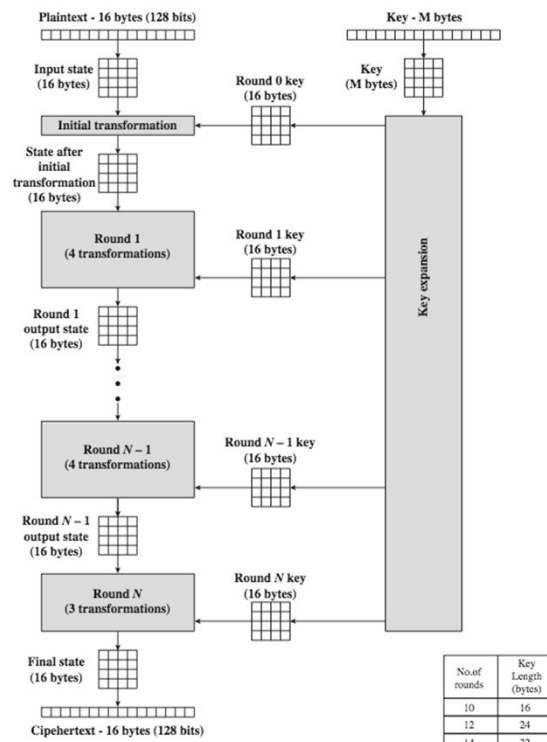
The Advanced Encryption Standard (AES) was published by NIST (National Institute of Standards and Technology) in 2001. AES is a symmetric block cipher that is intended to replace DES as the approved standard for a wide range of applications. The AES cipher (& other candidates) form the latest generation of block ciphers, and now we see a significant increase in the block size - from the old standard of 64-bits up to 128-bits; and keys from 128 to 256-bits. In part this has been driven by the public demonstrations of exhaustive key searches of DES. Whilst triple-DES is regarded as secure and well understood, it is slow, especially in s/w. In a first round of evaluation, 15 proposed algorithms were accepted. A second round narrowed the field to 5 algorithms. NIST completed its evaluation process and published a final standard (FIPS PUB 197) in November of 2001. NIST selected Rijndael as the proposed AES algorithm. The two researchers who developed and submitted Rijndael for the AES are both cryptographers from Belgium: Dr. Joan Daemen and Dr. Vincent Rijmen.

The AES Cipher - Rijndael

- ❑ designed by Rijmen-Daemen in Belgium
- ❑ has 128/192/256 bit keys, 128 bit data
- ❑ an **iterative** rather than **feistel** cipher
 - processes data as block of 4 columns of 4 bytes
 - operates on entire data block in every round
- ❑ designed to be:
 - resistant against known attacks
 - speed and code compactness on many CPUs
 - design simplicity

The Rijndael proposal for AES defined a cipher in which the block length and the key length can be independently specified to be 128, 192, or 256 bits. The AES specification uses the same three key size alternatives but limits the block length to 128 bits. Rijndael is an academic submission, based on the earlier Square cipher, from Belgium academics Dr Joan Daemen and Dr Vincent Rijmen. It is an iterative cipher (operates on entire data block in every round) rather than feistel (operate on halves at a time), and was designed to have characteristics of: Resistance against all known attacks, Speed and code compactness on a wide range of platforms, & Design simplicity.

AES Encryption Process



Stallings Figure 5.1 shows the overall encryption process in AES.

AES Structure

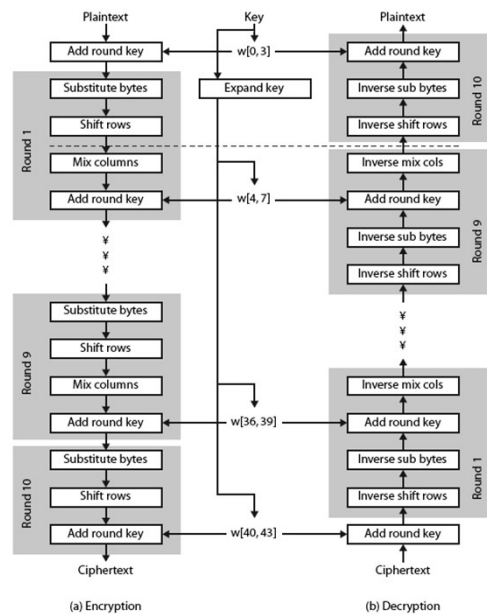
- data block of 4 columns of 4 bytes is state
- key is expanded to array of words
- has 9/11/13 rounds in which state undergoes:
 - byte substitution (1 S-box used on every byte)
 - shift rows (permute bytes between groups/columns)
 - mix columns (subs using matrix multiply of groups)
 - add round key (XOR state with key material)
 - view as alternating XOR key & scramble data bytes
- initial XOR key material & incomplete last round
- with fast XOR & table lookup implementation

The input to the AES encryption and decryption algorithms is a single 128-bit block, depicted in FIPS PUB 197, as a square matrix of bytes. This block is copied into the State array, which is modified at each stage of encryption or decryption. After the final stage, State is copied to an output.

The key is expanded into 44/52/60 lots of 32-bit words (see later), with 4 used in each round. Note that the ordering of bytes within a matrix is by column. So, for example, the first four bytes of a 128-bit plaintext input to the encryption cipher occupy the first column of the in matrix, the second four bytes occupy the second column, and so on. Similarly, the first four bytes of the expanded key, which form a word, occupy the first column of the w matrix.

The data computation then consists of an “add round key” step, then 9/11/13 rounds with all 4 steps, and a final 10th/12th/14th step of byte subs + mix cols + add round key. This can be viewed as alternating XOR key & scramble data bytes operations. All of the steps are easily reversed, and can be efficiently implemented using XOR’s & table lookups.

AES Structure



Stallings Figure 5.3 shows the structure of AES in more detail. The cipher consists of N rounds, where the number of rounds depends on the key length: 10 rounds for a 16-byte key; 12 rounds for a 24-byte key; and 14 rounds for a 32-byte key. The first $N - 1$ rounds consist of four distinct transformation functions: SubBytes, ShiftRows, MixColumns, and AddRoundKey, which are described subsequently. The final round contains only 3 transformations, and there is an initial single transformation (AddRoundKey) before the first round, which can be considered Round 0. Each transformation takes one or more 4×4 matrices as input and produces a 4×4 matrix as output. Figure 5.1 shows that the output of each round is a 4×4 matrix, with the output of the final round being the ciphertext. Also, the key expansion function generates $N + 1$ round keys, each of which is a distinct 4×4 matrix. Each round key serves as one of the inputs to the AddRoundKey transformation in each round.

Some Comments on AES

- ❑ an **iterative** rather than **feistel** cipher
- ❑ key expanded into array of 32-bit words
 - four words form round key in each round
- ❑ 4 different stages are used as shown
- ❑ has a simple structure
- ❑ only AddRoundKey uses key
- ❑ AddRoundKey a form of Vernam cipher
- ❑ each stage is easily reversible
- ❑ decryption uses keys in reverse order
- ❑ decryption does recover plaintext
- ❑ final round has only 3 stages

Before delving into details, can make several comments about the overall AES structure. See text for details.

Substitute Bytes

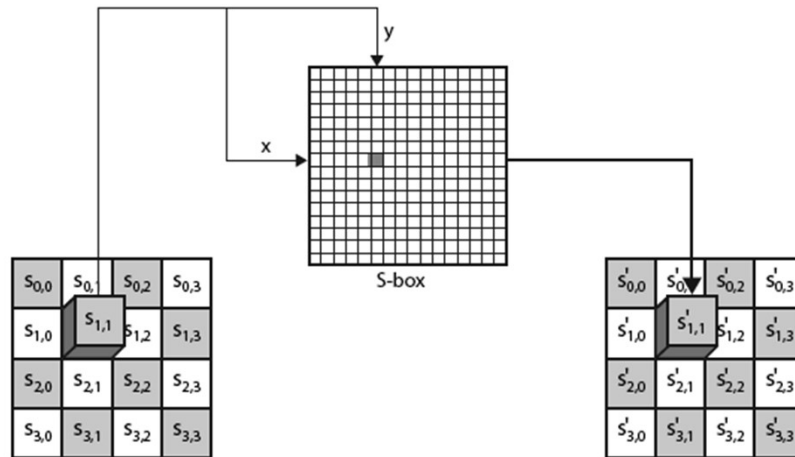
- ❑ a simple substitution of each byte
- ❑ uses one table of 16x16 bytes containing a permutation of all 256 8-bit values
- ❑ each byte of state is replaced by byte indexed by row (left 4-bits) & column (right 4-bits)
 - eg. byte {95} is replaced by byte in row 9 column 5
 - which has value {2A}
- ❑ S-box constructed using defined transformation of values in $GF(2^8)$
- ❑ designed to be resistant to all known attacks

We now turn to a discussion of each of the four transformations used in AES. For each stage, we mention the forward (encryption) algorithm, the inverse (decryption) algorithm, and the rationale for the design of that stage.

The Substitute bytes stage uses an S-box to perform a byte-by-byte substitution of the block. There is a single 8-bit wide S-box used on every byte. This S-box is a permutation of all 256 8-bit values, constructed using a transformation which treats the values as polynomials in $GF(2^8)$ – however it is fixed, so really only need to know the table when implementing. Decryption requires the inverse of the table. These tables are given in Stallings Table 5.2.

The table was designed to be resistant to known cryptanalytic attacks. Specifically, the Rijndael developers sought a design that has a low correlation between input bits and output bits, with the property that the output cannot be described as a simple mathematical function of the input, with no fixed points and no “opposite fixed points”.

Substitute Bytes



As this diagram from Stallings Fig 5.5a shows, the Byte Substitution operates on each byte of state independently, with the input byte used to index a row/col in the table to retrieve the substituted value.

Substitute Bytes Example

EA	04	65	85
83	45	5D	96
5C	33	98	B0
F0	2D	AD	C5

→

87	F2	4D	97
EC	6E	4C	90
4A	C3	46	E7
8C	D8	95	A6

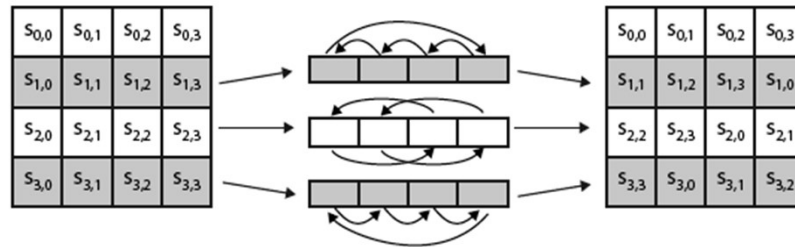
Show an example of the SubBytes transformation from the text.

Shift Rows

- ❑ a circular byte shift in each each
 - 1st row is unchanged
 - 2nd row does 1 byte circular shift to left
 - 3rd row does 2 byte circular shift to left
 - 4th row does 3 byte circular shift to left
- ❑ decrypt inverts using shifts to right
- ❑ since state is processed by columns, this step permutes bytes between the columns

The ShiftRows stage provides a simple “permutation” of the data, whereas the other steps involve substitutions. Further, since the state is treated as a block of columns, it is this step which provides for diffusion of values between columns. It performs a circular rotate on each row of 0, 1, 2 & 3 places for respective rows. When decrypting it performs the circular shifts in the opposite direction for each row. This row shift moves an individual byte from one column to another, which is a linear distance of a multiple of 4 bytes, and ensures that the 4 bytes of one column are spread out to four different columns.

Shift Rows



87	F2	4D	97
EC	6E	4C	90
4A	C3	46	E7
8C	D8	95	A6

→

87	F2	4D	97
6E	4C	90	EC
46	E7	4A	C3
A6	8C	D8	95

Stalling Figure 5.7a illustrates the Shift Rows permutation. Then show an example of ShiftRows from the text.

Mix Columns

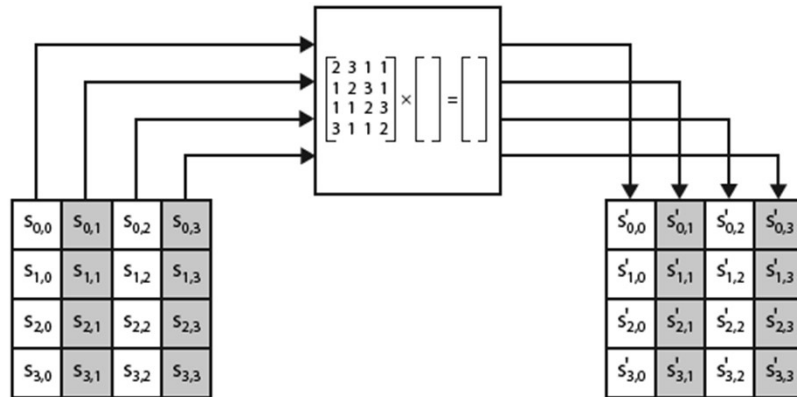
- ❑ each column is processed separately
- ❑ each byte is replaced by a value dependent on all 4 bytes in the column
- ❑ effectively a matrix multiplication in $GF(2^8)$ using prime poly $m(x) = x^8 + x^4 + x^3 + x + 1$

$$\begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix} \begin{bmatrix} s_{0,0} & s_{0,1} & s_{0,2} & s_{0,3} \\ s_{1,0} & s_{1,1} & s_{1,2} & s_{1,3} \\ s_{2,0} & s_{2,1} & s_{2,2} & s_{2,3} \\ s_{3,0} & s_{3,1} & s_{3,2} & s_{3,3} \end{bmatrix} = \begin{bmatrix} \dot{s}_{0,0} & \dot{s}_{0,1} & \dot{s}_{0,2} & \dot{s}_{0,3} \\ \dot{s}_{1,0} & \dot{s}_{1,1} & \dot{s}_{1,2} & \dot{s}_{1,3} \\ \dot{s}_{2,0} & \dot{s}_{2,1} & \dot{s}_{2,2} & \dot{s}_{2,3} \\ \dot{s}_{3,0} & \dot{s}_{3,1} & \dot{s}_{3,2} & \dot{s}_{3,3} \end{bmatrix}$$

The forward mix column transformation, called MixColumns, operates on each column individually. Each byte of a column is mapped into a new value that is a function of all four bytes in that column. It is a substitution that makes use of arithmetic over $GF(2^8)$. Each byte of a column is mapped into a new value that is a function of all four bytes in that column. It is designed as a matrix multiplication where each byte is treated as a polynomial in $GF(2^8)$. The inverse used for decryption involves a different set of constants.

The constants used are based on a linear code with maximal distance between code words – this gives good mixing of the bytes within each column. Combined with the “shift rows” step provides good avalanche, so that within a few rounds, all output bits depend on all input bits.

Mix Columns



Stalling Figure 5.5b illustrates the Mix Columns transformation.

Mix Columns Example

87	F2	4D	97
6E	4C	90	EC
46	E7	4A	C3
A6	8C	D8	95

→

47	40	A3	4C
37	D4	70	9F
94	E4	3A	42
ED	A5	A6	BC

$$(\{02\} \cdot \{87\}) \oplus (\{03\} \cdot \{6E\}) \oplus \{46\} \oplus \{A6\} = \{47\}$$

$$\{87\} \oplus (\{02\} \cdot \{6E\}) \oplus (\{03\} \cdot \{46\}) \oplus \{A6\} = \{37\}$$

$$\{87\} \oplus \{6E\} \oplus (\{02\} \cdot \{46\}) \oplus (\{03\} \cdot \{A6\}) = \{94\}$$

$$(\{03\} \cdot \{87\}) \oplus \{6E\} \oplus \{46\} \oplus (\{02\} \cdot \{A6\}) = \{ED\}$$

Show an example of the MixColumns transformation from the text, along with verification of the first column of this example.

AES Arithmetic

□ uses arithmetic in the finite field $GF(2^8)$

□ with irreducible polynomial

$$m(x) = x^8 + x^4 + x^3 + x + 1$$

which is (100011011) or {11b}

□ e.g.

$$\{02\} \cdot \{87\} \bmod \{11b\} = (1\ 0000\ 1110) \bmod \{11b\}$$

$$= (1\ 0000\ 1110) \text{ xor } (1\ 0001\ 1011) = (0001\ 0101)$$

AES uses arithmetic in the finite field $GF(2^8)$, with the irreducible polynomial $m(x) = x^8 + x^4 + x^3 + x + 1$. AES operates on 8-bit bytes. Addition of two bytes is defined as the bitwise XOR operation. Multiplication of two bytes is defined as multiplication in the finite field $GF(2^8)$. In particular, multiplication of a value by x (i.e., by {02}) can be implemented as a 1-bit left shift followed by a conditional bitwise XOR with (0001 1011) if the leftmost bit of the original value (prior to the shift) is 1.

Mix Columns

- ❑ can express each col as 4 equations
 - to derive each new byte in col
- ❑ decryption requires use of inverse matrix
 - with larger coefficients, hence a little harder
- ❑ have an alternate characterisation
 - each column a 4-term polynomial
 - with coefficients in $GF(2^8)$
 - and polynomials multiplied modulo (x^4+1)
- ❑ coefficients based on linear code with maximal distance between codewords

In practise, you implement Mix Columns by expressing the transformation on each column as 4 equations (Stallings equation 5.4) to compute the new bytes for that column. This computation only involves shifts, XORs & conditional XORs (for the modulo reduction).

The decryption computation requires the use of the inverse of the matrix, which has larger coefficients, and is thus potentially a little harder & slower to implement.

The designers & the AES standard provide an alternate characterisation of Mix Columns, which treats each column of State to be a four-term polynomial with coefficients in $GF(2^8)$. Each column is multiplied by a fixed polynomial $a(x)$ given in Stallings eqn 5.7. Whilst this is useful for analysis of the stage, the matrix description is all that's required for implementation.

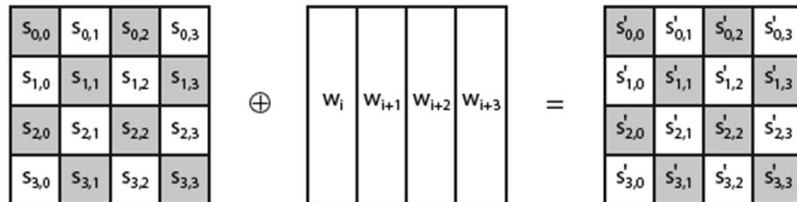
The coefficients of the matrix are based on a linear code with maximal distance between code words, which ensures a good mixing among the bytes of each column. The mix column transformation combined with the shift row transformation ensures that after a few rounds, all output bits depend on all input bits. In addition, the choice of coefficients in MixColumns, which are all {01}, {02}, or {03}, was influenced by implementation considerations.

Add Round Key

- XOR state with 128-bits of the round key
- again processed by column (though effectively a series of byte operations)
- inverse for decryption identical
 - since XOR own inverse, with reversed keys
- designed to be as simple as possible
 - a form of Vernam cipher on expanded key
 - requires other stages for complexity / security

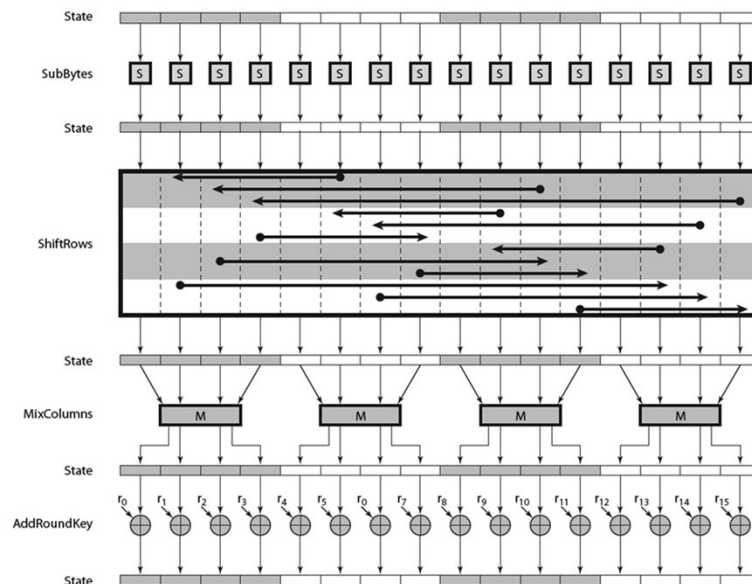
Lastly is the Add Round Key stage which is a simple bitwise XOR of the current block with a portion of the expanded key. Note this is the only step which makes use of the key and obscures the result, hence MUST be used at start and end of each round, since otherwise could undo effect of other steps. But the other steps provide confusion/diffusion/non-linearity. That us you can look at the cipher as a series of XOR with key then scramble/permute block repeated. This is efficient and highly secure it is believed.

Add Round Key



Stallings Figure 5.5b illustrates the Add Round Key stage, which like Byte Substitution, operates on each byte of state independently.

AES Round



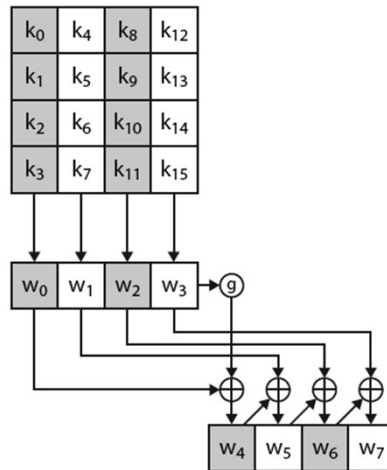
Can thus now view all the internal details of the AES round, showing how each byte of the state is manipulated, as shown in Stallings Figure 5.4.

AES Key Expansion

- takes 128-bit (16-byte) key and expands into array of 44/52/60 32-bit words
- start by copying key into first 4 words
- then loop creating words that depend on values in previous & 4 places back
 - in 3 of 4 cases just XOR these together
 - 1st word in 4 has rotate + S-box + XOR round constant on previous, before XOR 4th back

The AES key expansion algorithm takes as input a 4-word (16-byte) key and produces a linear array of words, providing a 4-word round key for the initial AddRoundKey stage and each of the 10/12/14 rounds of the cipher. It involves copying the key into the first group of 4 words, and then constructing subsequent groups of 4 based on the values of the previous & 4th back words. The first word in each group of 4 gets “special treatment” with rotate + S-box + XOR constant on the previous word before XOR’ing the one from 4 back. In the 256-bit key/14 round version, there’s also an extra step on the middle word. The text includes in section 5.4 pseudocode that describes the key expansion.

AES Key Expansion



The first block of the AES Key Expansion is shown here in Stallings Figure 5.9a. It shows each group of 4 bytes in the key being assigned to the first 4 words, then the calculation of the next 4 words based on the values of the previous 4 words, which is repeated enough times to create all the necessary subkey information.

Key Expansion Rationale

- ❑ designed to resist known attacks
- ❑ design criteria included
 - knowing part key insufficient to find many more
 - invertible transformation
 - fast on wide range of CPU's
 - use round constants to break symmetry
 - diffuse key bits into round keys
 - enough non-linearity to hinder analysis
 - simplicity of description

The Rijndael developers designed the expansion key algorithm to be resistant to known cryptanalytic attacks. It is designed to be simple to implement, but by using round constants break symmetries, and make it much harder to deduce other key bits if just some are known (but once have as many consecutive bits as are in key, can then easily recreate the full expansion). The design criteria used are listed above.

AES Example Key Expansion

Key Words	Auxiliary Function
w0 = 0f 15 71 c9	RotWord(w3)= 7f 67 98 af = x1
w1 = 47 d9 e8 59	SubWord(x1)= d2 85 46 79 = y1
w2 = 0c b7 ad	Rcon(1)= 01 00 00 00
w3 = af 7f 67 98	y1 ⊕ Rcon(1)= d3 85 46 79 = z1
w4 = w0 ⊕ z1 = dc 90 37 b0	RotWord(w7)= 81 15 a7 38 = x2
w5 = w4 ⊕ w1 = 9b 49 df e9	SubWord(x4)= 0c 59 5c 07 = y2
w6 = w5 ⊕ w2 = 97 fe 72 3f	Rcon(2)= 02 00 00 00
w7 = w6 ⊕ w3 = 38 81 15 a7	y2 ⊕ Rcon(2)= 0e 59 5c 07 = z2
w8 = w4 ⊕ z2 = d2 c9 6b b7	RotWord(w11)= ff d3 c6 e6 = x3
w9 = w8 ⊕ w5 = 49 80 b4 5e	SubWord(x2)= 16 66 b4 8e = y3
w10 = w9 ⊕ w6 = de 7e c6 61	Rcon(3)= 04 00 00 00
w11 = w10 ⊕ w7 = e6 ff d3 c6	y3 ⊕ Rcon(3)= 12 66 b4 8e = z3
w12 = w8 ⊕ z3 = c0 af df 39	RotWord(w15)= ae 7e c0 b1 = x4
w13 = w12 ⊕ w9 = 89 2f 6b 67	SubWord(x3)= e4 f3 ba c8 = y4
w14 = w13 ⊕ w10 = 57 51 ad 06	Rcon(4)= 08 00 00 00
w15 = w14 ⊕ w11 = b1 ae 7e c0	y4 ⊕ Rcon(4)= ec f3 ba c8 = 4
w16 = w12 ⊕ z4 = 2c 5c 65 f1	RotWord(w19)= 8c dd 50 43 = x5
w17 = w16 ⊕ w13 = a5 73 0e 96	SubWord(x4)= 64 c1 53 1a = y5
w18 = w17 ⊕ w14 = f2 22 a3 90	Rcon(5)= 10 00 00 00
w19 = w18 ⊕ w15 = 43 8c dd 50	y5 ⊕ Rcon(5)= 74 c1 53 1a = z5
w20 = w16 ⊕ z5 = 58 9d 36 eb	RotWord(w23)= 40 46 bd 4c = x6
w21 = w20 ⊕ w17 = fd ee 38 7d	SubWord(x5)= 09 5a 7a 29 = y6
w22 = w21 ⊕ w18 = 0f cc 9b ed	Rcon(6)= 20 00 00 00
w23 = w22 ⊕ w19 = 4c 40 46 bd	y6 ⊕ Rcon(6)= 29 5a 7a 29 = z6
w24 = w20 ⊕ z6 = 71 c7 4c c2	RotWord(w27)= a5 a9 ef cf = x7
w25 = w24 ⊕ w21 = 8c 29 74 bf	SubWord(x6)= 06 d3 df 8a = y7
w26 = w25 ⊕ w22 = 83 e5 ef 52	Rcon(7)= 40 00 00 00
w27 = w26 ⊕ w23 = cf a5 a9 ef	y7 ⊕ Rcon(7)= 46 d3 df 8a = z7
w28 = w24 ⊕ z7 = 37 14 93 48	RotWord(w31)= 7d a1 4a f7 = x8
w29 = w28 ⊕ w25 = bb 3d e7 f7	SubWord(x7)= ff 32 d6 68 = y8
w30 = w29 ⊕ w26 = 38 d8 08 a5	Rcon(8)= 80 00 00 00
w31 = w30 ⊕ w27 = f7 7d a1 4a	y8 ⊕ Rcon(8)= 7f 32 d6 68 = z8
w32 = w28 ⊕ z8 = 48 26 45 20	RotWord(w35)= be 0b 38 3c = x9
w33 = w32 ⊕ w29 = f3 1b a2 d7	SubWord(x8)= ae 2b 07 eb = y9
w34 = w33 ⊕ w30 = cb c3 aa 72	Rcon(9)= 18 00 00 00
w35 = w34 ⊕ w32 = 3c be 0b 38	y9 ⊕ Rcon(9)= b5 2b 07 eb = z9
w36 = w32 ⊕ z9 = fd 0d 42 cb	RotWord(w39)= 6b 41 56 f9 = x10
w37 = w36 ⊕ w33 = 0e 16 e0 1c	SubWord(x9)= 7f 83 b1 99 = y10
w38 = w37 ⊕ w34 = c5 d5 4a 6e	Rcon(10)= 36 00 00 00
w39 = w38 ⊕ w35 = f9 6b 41 56	y10 ⊕ Rcon(10)= 49 83 b1 99 = z10
w40 = w36 ⊕ z10 = b4 8e f3 52	
w41 = w40 ⊕ w37 = ba 98 13 4e	
w42 = w41 ⊕ w38 = 7f 4d 59 20	
w43 = w42 ⊕ w39 = 86 26 18 76	

We now work through an example, and consider some of its implications. The plaintext, key, and resulting ciphertext are as follows:

Plaintext: 0123456789abcdeffedcba9876543210

Key: 0f1571c947d9e8590cb7add6af7f6798

Ciphertext: ff0b844a0853bf7c6934ab4364148fb9

Table 5.3 shows the expansion of the 16-byte key into 10 round keys. As previously explained, this process is performed word by word, with each four-byte word occupying one column of the word round key matrix. The left hand column shows the four round key words generated for each round. The right hand column shows the steps used to generate the auxiliary word used in key expansion. We begin, of course, with the key itself serving as the round key for round 0.

AES Example Encryption

Start of round	After SubBytes	After ShiftRows	After MixColumns	Round Key
01 89 fe 76 23 ab dc 54 45 cd ba 32 67 ef 98 10				0f 47 0c af 15 d9 b7 7f 71 e8 ad 67 c9 59 d6 98
0e ce f2 d9 36 72 6b 2b 34 25 17 55 ae b6 4e 88	ab 8b 89 35 05 40 7f f1 18 3f f0 fc e4 4e 2f c4	ab 8b 89 35 40 7f f1 05 f0 fc 18 3f c4 e4 4e 2f	b9 94 57 75 e4 8e 16 51 47 20 9a 3f c5 d6 f5 3b	dc 9b 97 38 90 49 fe 81 37 df 72 15 b0 e9 3f a7
65 0f c0 4d 74 c7 e8 d0 70 ff e8 2a 75 3f ca 9c	4d 76 ba e3 92 c6 9b 70 51 16 9b e5 9d 75 74 de	4d 76 ba e3 c6 9b 70 92 9b e5 51 16 de 9d 75 74	8e 22 db 12 b2 f2 dc 92 df 80 f7 c1 2d c5 1e 52	d2 49 de e6 c9 80 7e ff 6b b4 c6 d3 b7 5e 61 c6
5c 6b 05 f4 7b 72 a2 6d b4 34 31 12 9a 9b 7f 94	4a 7f 6b bf 21 40 3a 3c 8d 18 c7 c9 b8 14 d2 22	4a 7f 6b bf 40 3a 3c 21 c7 c9 8d 18 22 b8 14 d2	b1 c1 0b cc ba f3 8b 07 f9 1f 6a c3 1d 19 24 5c	c0 89 57 b1 af 2f 51 ae df 6b ad 7e 39 67 06 c0
71 48 5c 7d 15 dc da a9 26 74 c7 bd 24 7e 22 9c	a3 52 4a ff 59 86 57 d3 f7 92 c6 7a 36 f3 93 de	a3 52 4a ff 86 57 d3 59 c6 7a f7 92 de 36 f3 93	d4 11 fe 0f 3b 44 06 73 cb ab 62 37 19 b7 07 ec	2c a5 f2 43 5c 73 22 8c 65 0e a3 dd f1 96 90 50
f8 b4 0c 4c 67 37 24 ff ae a5 c1 ea e8 21 97 bc	41 8d fe 29 85 9a 36 16 e4 06 78 87 9b fd 88 65	41 8d fe 29 9a 36 16 85 78 87 e4 06 65 9b fd 88	2a 47 c4 48 83 e8 18 ba 84 18 27 23 eb 10 0a f3	58 fd 0f 4c 9d ee cc 40 36 38 9b 46 eb 7d ed bd
72 ba cb 04 1e 06 d4 fa b2 20 bc 65 00 6d e7 4e	40 f4 1f f2 72 6f 48 2d 37 b7 65 4d 63 3c 94 2f	40 f4 1f f2 6f 48 2d 72 65 4d 37 b7 2f 63 3c 94	7b 05 42 aa 1e d0 20 40 94 83 18 52 94 c4 43 fb	71 8c 83 cf c7 29 e5 a5 4c 74 ef a9 c2 bf 52 ef
0a 89 c1 85 d9 f9 c5 e5 d8 f7 f7 fb 56 7b 11 14	67 a7 78 97 35 99 a6 d9 61 68 68 0f b1 21 82 fa	67 a7 78 97 99 a6 d9 35 68 0f 61 68 fa b1 21 82	ec 1a c0 80 0c 50 53 c7 3b d7 00 ef b7 22 72 e0	37 bb 38 f7 14 3d d8 7d 93 e7 08 a1 48 f7 a5 4a
db a1 f8 77 18 6d 8b ba a8 30 08 4e ff d5 d7 aa	b9 32 41 f5 ad 3c 3d f4 c2 04 30 2f 16 03 0e ac	b9 32 41 f5 3c 3d f4 ad 30 2f c2 04 ac 16 03 0e	b1 1a 44 17 3d 2f ec b6 0a 6b 2f 42 9f 68 f3 b1	48 f3 cb 3c 26 1b c3 be 45 a2 aa 0b 20 d7 72 38
e9 e9 8f 2b 1b 34 2f 08 4f c9 85 49 bf bf 81 89	99 1e 73 f1 af 18 15 30 84 dd 97 3b 08 08 0c a7	99 1e 73 f1 18 15 30 af 97 3b 84 dd a7 08 08 0c	31 30 3a c2 ac 71 8c c4 46 65 48 eb 6a 1c 31 62	fd 0e c5 f9 0d 16 d5 6b 42 e0 4a 41 cb 1c 6e 56
cc 3e ff 3b a1 67 59 af 04 85 02 aa a1 00 5f 34	4b b2 16 e2 32 85 cb 79 f2 97 77 ac 32 63 cf 18	4b b2 16 e2 85 cb 79 32 77 ac f2 97 18 32 63 cf	4b 86 8a 36 b1 cb 27 5a fb f2 f2 af cc 5a 5b cf	b4 8e f3 52 ba 98 13 4e 7f 4d 59 20 86 26 18 76
ff 08 69 64 0b 53 34 14 84 bf ab 8f 4a 7c 43 b9				

Next, Table 5.4 shows the progression of the state matrix through the AES encryption process. The first column shows the value of the state matrix at the start of a round. For the first row, the state matrix is just the matrix arrangement of the plaintext. The second, third, and fourth columns show the value of the state matrix for that round after the SubBytes, ShiftRows, and MixColumns transformations, respectively. The fifth column shows the round key. You can verify that these round keys equate with those shown in Table 5.3. The first column shows the value of the state matrix resulting from the bitwise XOR of the state after the preceding MixColumns with the round key for the preceding round.

AES Example Avalanche

Round		Number of bits that differ
	0123456789abcdef fedcba9876543210 0023456789abcdef fedcba9876543210	1
0	0e3634aece7225b6f26b174ed92b5588 0f3634aece7225b6f26b174ed92b5588	1
1	657470750fc7ff3fc0e8e8ca4dd02a9c c4a9ad090fc7ff3fc0e8e8ca4dd02a9c	20
2	5c7bb49a6b72349b05a2317ff46d1294 fe2ae569f7ee8bb8c1f5a2bb37ef53d5	58
3	7115262448dc747e5cdac7227da9bd9c ec093dfb7c45343d689017507d485e62	59
4	f867aee8b437a5210c24c1974cffeabc 43efdb697244df808e8d9364ee0ae6f5	61
5	721eb200ba06206dcbd4bce704fa654e 7b28a5d5ed643287e006c099bb375302	68
6	0ad9d85689f9f77bc1c5f71185e5fb14 3bc2d8b6798d8ac4fe36a1d891ac181a	64
7	db18a8ffa16d30d5f88b08d777ba4eaa 9fb8b5452023c70280e5c4bb9e555a4b	67
8	f91b4fbfe934c9bf8f2f85812b084989 20264e1126b219aef7feb3f9b2d6de40	65
9	cca104a13e678500ff59025f3bafaa34 b56a0341b2290ba7dfdfbddcd8578205	61
10	ff0b844a0853bf7c6934ab4364148fb9 612b89398d0600cde116227ce72433f0	58

In any good cipher design, want the avalanche effect, in which a small change in plaintext or key produces a large change in the ciphertext. Using the example from Table 5.4, Table 5.5 shows the result when the eighth bit of the plaintext is changed. The second column of the table shows the value of the state matrix at the end of each round for the two plaintexts. Note that after just one round, 20 bits of the state vector differ. And after two rounds, close to half the bits differ. This magnitude of difference propagates through the remaining rounds. A bit difference in approximately half the positions in the most desirable outcome.

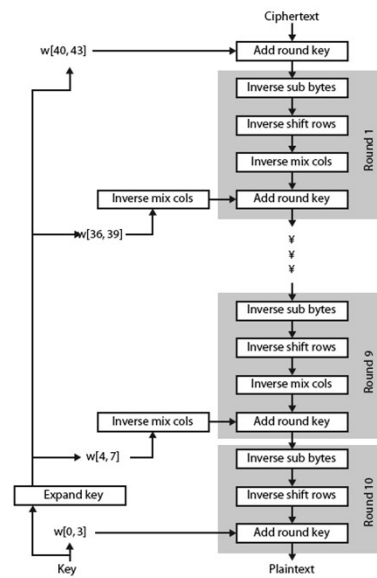
AES Decryption

- ❑ AES decryption is not identical to encryption since steps done in reverse
- ❑ but can define an equivalent inverse cipher with steps as for encryption
 - but using inverses of each step
 - with a different key schedule
- ❑ works since result is unchanged when
 - swap byte substitution & shift rows
 - swap mix columns & add (tweaked) round key

The AES decryption cipher is not identical to the encryption cipher (Stallings Figure 5.3). The sequence of transformations for decryption differs from that for encryption, although the form of the key schedules for encryption and decryption is the same. This has the disadvantage that two separate software or firmware modules are needed for applications that require both encryption and decryption. There is, however, an equivalent version of the decryption algorithm that has the same structure as the encryption algorithm, with the same sequence of transformations as the encryption algorithm (with transformations replaced by their inverses). To achieve this equivalence, a change in key schedule is needed.

By constructing an equivalent inverse cipher with steps in same order as for encryption, we can derive a more efficient implementation. Clearly swapping the byte substitutions and shift rows has no effect, since work just on bytes. Swapping the mix columns and add round key steps requires the inverse mix columns step be applied to the round keys first – this makes the decryption key schedule a little more complex with this construction, but allows the use of same h/w or s/w for the data en/decrypt computation.

AES Decryption



Illustrate the equivalent inverse cipher with Stallings Figure 5.10.

Implementation Aspects

- ❑ can efficiently implement on 8-bit CPU
 - byte substitution works on bytes using a table of 256 entries
 - shift rows is simple byte shift
 - add round key works on byte XOR's
 - mix columns requires matrix multiply in $GF(2^8)$ which works on byte values, can be simplified to use table lookups & byte XOR's

The Rijndael proposal [DAEM99] provides some suggestions for efficient implementation on 8-bit processors, typical for current smart cards, and on 32-bit processors, typical for PCs.

AES can be implemented very efficiently on an 8-bit processor.

- AddRoundKey is a bitwise XOR operation.
- ShiftRows is a simple byte shifting operation.
- SubBytes operates at the byte level and only requires a lookup of a 256 byte table S.
- MixColumns (matrix multiply) can be implemented as byte XOR's & table lookups with a 2nd 256 byte table X2, using the formulae shown in Stallings equation 5.9.

Implementation Aspects

- can efficiently implement on 32-bit CPU
 - redefine steps to use 32-bit words
 - can precompute 4 tables of 256-words
 - then each column in each round can be computed using 4 table lookups + 4 XORs
 - at a cost of 4Kb to store tables
- designers believe this very efficient implementation was a key factor in its selection as the AES cipher

AES can also be very efficiently implemented on an 32-bit processor, by rewriting the stage transformation to use 4 table lookups & 4 XOR's per column of state. These tables can be computed in advance using the formulae shown in the text, and need 4Kb to store.

The developers of Rijndael believe that this compact, efficient implementation was probably one of the most important factors in the selection of Rijndael for AES.

Summary

- have considered:
 - the AES selection process
 - the details of Rijndael - the AES cipher
 - looked at the steps in each round
 - the key expansion
 - implementation aspects

Chapter 5 summary.